

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions
– Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I Moshitha Kolanjiyappan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Moshitha K

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Aim

To implement KNN using Euclidean distance and classify a test instance based on the majority class of its nearest neighbours.

Concept

KNN is a supervised learning algorithm used for classification and regression. It classifies a new data point based on the classes of its nearest neighbours.

Euclidean Distance Formula

$$\text{Distance} = \sqrt{[\sum(x_i - y_i)^2]}$$

Algorithm

1. Read the training dataset.
2. Read the value of K.
3. Calculate Euclidean distance between the test point and every training point.
4. Sort the distances in ascending order.
5. Select the K nearest points.
6. Find the majority class among the selected neighbours.
7. Predict the class of the test instance.
8. Calculate classification accuracy.

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
on darwin
Enter "help" below or click "Help" above for more information.

===== RESTART: /Users/moshitha/Documents/dd.py =====
Enter K: 3
Test Point: [2, 2] Predicted Class: 0
Test Point: [7, 7] Predicted Class: 1
Accuracy: 100.0 %

>>>
```

```
dd.py (3.13.14)
import math
from collections import Counter

X = [[1, 1], [2, 2], [3, 3], [6, 6], [7, 7], [8, 8]]
y = [0, 0, 0, 1, 1, 1]

def knn_predict(X, y, test, k):
    distances = []
    for i in range(len(X)):
        d = math.sqrt(sum((X[i][j] - test[j]) ** 2
                           for j in range(len(test))))
        distances.append((d, y[i]))
    distances.sort()
    neighbours = distances[:k]
    classes = [label for _, label in neighbours]
    return Counter(classes).most_common(1)[0][0]

k = int(input("Enter K: "))
test_points = [[2, 2], [7, 7]]
actual = [0, 1]
predicted = []

for point in test_points:
    p = knn_predict(X, y, point, k)
    predicted.append(p)
    print("Test Point:", point, "Predicted Class:", p)

accuracy = sum(p == a for p, a in zip(predicted, actual)) / len(actual)
print("Accuracy:", accuracy * 100, "%")
```

Analysis

KNN is a lazy learning algorithm because it does not build an explicit model during training. A smaller value of K can be sensitive to noise, while a larger value of K provides smoother classification.

Result

Thus, the KNN algorithm successfully classifies test instances using Euclidean distance and majority voting.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim

To implement Locally Weighted Regression using a Gaussian kernel and compare its prediction with standard Linear Regression.

Concept

Locally Weighted Regression is a non-parametric regression technique. It assigns higher weights to data points that are closer to the query point and lower weights to distant points.

Gaussian Weight Formula

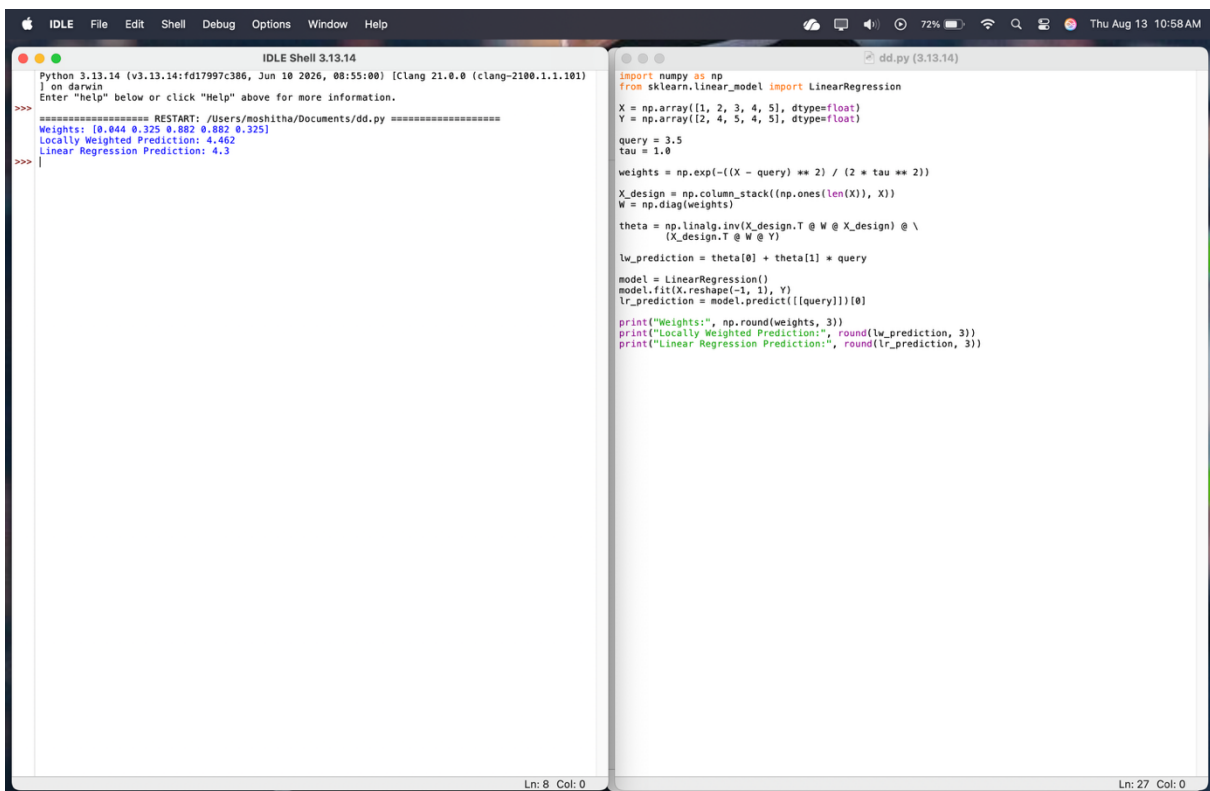
$$w_i = \exp(-(x_i - x_q)^2 / 2\tau^2)$$

where:

- x_i = training data point
- x_q = query point
- τ = bandwidth parameter

Algorithm

1. Read the dataset.
2. Select a query point.
3. Calculate the distance between each training point and the query point.
4. Calculate Gaussian weights using the distance.
5. Construct the weighted regression model.
6. Predict the output for the query point.
7. Train a standard Linear Regression model.
8. Compare both predictions.



```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 00:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
>>>
===== RESTART: /Users/moshitha/Documents/dd.py =====
Weights: [0.844 0.325 0.882 0.882 0.325]
Locally Weighted Prediction: 4.462
Linear Regression Prediction: 4.3
>>>

import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 5, 4, 5], dtype=float)

query = 3.5
tau = 1.0

weights = np.exp(-(X - query) ** 2 / (2 * tau ** 2))

X_design = np.column_stack((np.ones(len(X)), X))
W = np.diag(weights)

theta = np.linalg.inv(X_design.T @ W @ X_design) @ \
        (X_design.T @ W @ Y)

lw_prediction = theta[0] + theta[1] * query

model = LinearRegression()
model.fit(X.reshape(-1, 1), Y)
lr_prediction = model.predict([[query]])[0]

print("Weights:", np.round(weights, 3))
print("Locally Weighted Prediction:", round(lw_prediction, 3))
print("Linear Regression Prediction:", round(lr_prediction, 3))
```

Analysis

Locally Weighted Regression focuses more on nearby observations. The bandwidth parameter τ controls the locality. A smaller τ gives more importance to nearby points, while a larger τ produces a smoother model.

Result

Thus, Locally Weighted Regression provides a local prediction and can behave differently from standard Linear Regression depending on the query point and bandwidth.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Aim

To implement a simple RBF network using Gaussian activation functions for regression or classification.

Concept

An RBF network is a type of artificial neural network that uses radial basis functions as activation functions. Gaussian functions are commonly used as RBF activation units.

Gaussian RBF Formula

$$\phi(x) = \exp(-(x - c)^2 / 2\sigma^2)$$

where:

- c = centre of the RBF unit
- σ = width of the Gaussian function

Algorithm

1. Prepare the training dataset.
2. Select the centres of the RBF units.
3. Select the width of each Gaussian function.
4. Calculate the RBF activation for each input.
5. Construct the activation matrix.
6. Train the output weights.
7. Generate predictions for the test data.
8. Compare predicted and actual outputs.

The screenshot shows an IDE with two windows. The left window, titled 'IDLE Shell 3.13.14', displays the output of a Python script. The right window, titled 'dd.py (3.13.14)', shows the source code of the script.

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/moshitha/Documents/dd.py =====
Centers: [1. 3.]
Width: 1.0
Predicted Outputs:
0.0 -> 3.583
1.0 -> -1.639
2.0 -> 4.309
3.0 -> 11.261
4.0 -> 12.466
>>>
```

```
import numpy as np

X = np.array([0, 1, 2, 3, 4], dtype=float)
Y = np.array([0, 1, 4, 9, 16], dtype=float)
centers = np.array([1, 3], dtype=float)
sigma = 1.0

def rbf(x, c, sigma):
    return np.exp(-((x - c) ** 2) / (2 * sigma ** 2))

Phi = np.array([
    [rbf(x, c, sigma) for c in centers]
    for x in X
])

Phi = np.column_stack((Phi, np.ones(len(X))))
weights = np.linalg.pinv(Phi) @ Y
predicted = Phi @ weights

print("Centers:", centers)
print("Width:", sigma)
print("Predicted Outputs:")

for x, p in zip(X, predicted):
    print(x, "->", round(p, 3))
```

Analysis

The centres determine the regions where the RBF units respond strongly. Smaller widths produce localized responses, while larger widths produce smoother responses. Proper selection of centres and widths improves model performance.

Result

Thus, the RBF network successfully uses Gaussian activation functions to model the given dataset.

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Aim

To implement a basic Case-Based Learning system that retrieves the most similar previous case for a new query.

Concept

Case-Based Learning solves a new problem by comparing it with previously stored cases. The solution of the most similar previous case is used for the new problem.

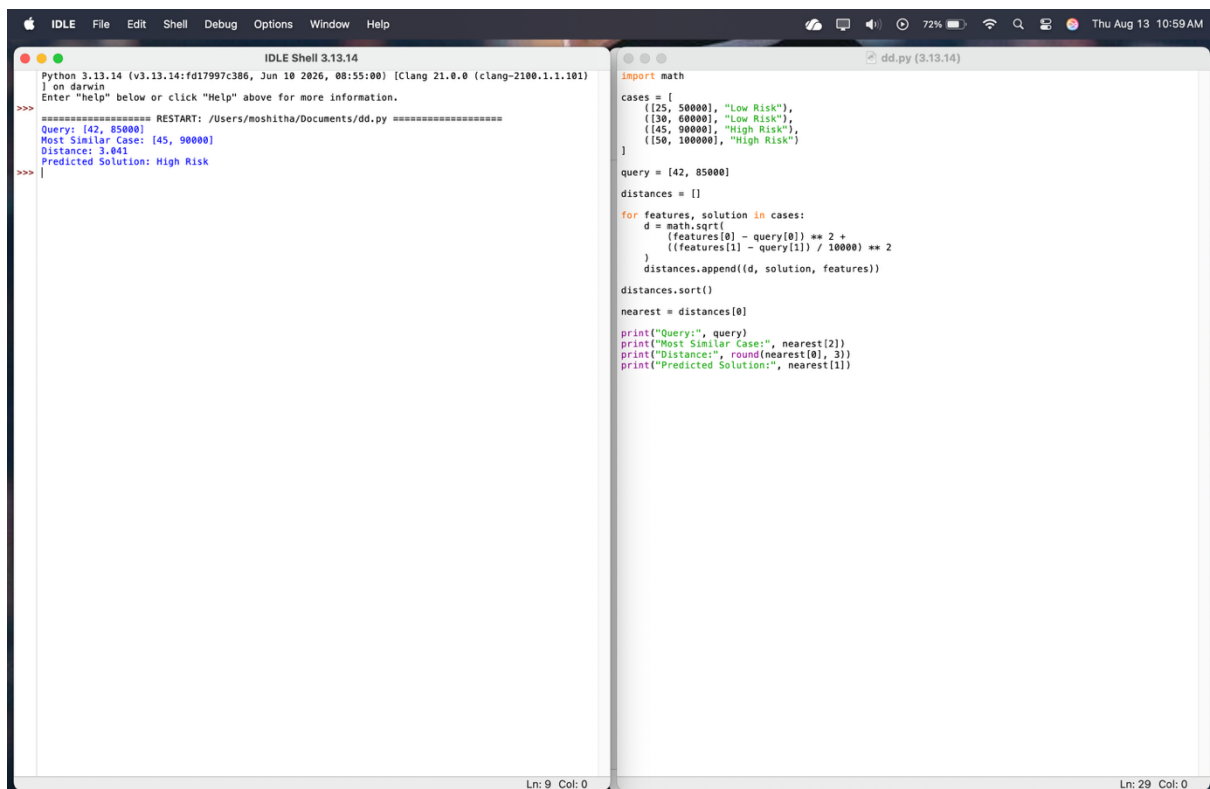
Similarity / Distance Formula

$$d = \sqrt{\sum (x_i - y_i)^2}$$

A smaller distance indicates greater similarity.

Algorithm

1. Store previous cases along with their solutions.
2. Accept a new query case.
3. Calculate the distance between the query and every stored case.
4. Compare the calculated distances.
5. Select the case having the minimum distance.
6. Retrieve its associated solution.
7. Display the most similar case and predicted solution.



The screenshot shows an IDE with two windows. The left window, titled 'IDLE Shell 3.13.14', displays the output of a Python script. The right window, titled 'dd.py (3.13.14)', shows the source code of the script.

Left Window Output:

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/moshitha/Documents/dd.py =====
Query: [42, 85000]
Most Similar Case: [45, 90000]
Distance: 3.041
Predicted Solution: High Risk
>>>
```

Right Window Code (dd.py):

```
import math

cases = [
    ([25, 50000], "Low Risk"),
    ([30, 60000], "Low Risk"),
    ([45, 90000], "High Risk"),
    ([50, 100000], "High Risk")
]

query = [42, 85000]
distances = []

for features, solution in cases:
    d = math.sqrt(
        (features[0] - query[0]) ** 2 +
        ((features[1] - query[1]) / 10000) ** 2
    )
    distances.append((d, solution, features))

distances.sort()
nearest = distances[0]

print("Query:", query)
print("Most Similar Case:", nearest[2])
print("Distance:", round(nearest[0], 3))
print("Predicted Solution:", nearest[1])
```

Analysis

Case-Based Learning depends on the quality of previous cases. A suitable distance measure is important because it determines which stored case is considered most similar.

Result

Thus, the Case-Based Learning system successfully retrieves the closest previous case and uses its solution for the new query.

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

Aim

To implement KNN and RBF models on the same dataset and compare their predictions, computation, accuracy, and output behaviour.

Concept

KNN predicts the class of a test point using nearby training examples, whereas an RBF model uses Gaussian functions to learn a decision boundary or regression function.

Algorithm

1. Prepare a common dataset.
2. Train the KNN model.
3. Select the value of K.
4. Train the RBF model.
5. Provide the same test inputs to both models.
6. Obtain predictions from both models.
7. Calculate and compare accuracy.
8. Analyze the differences in output behaviour.

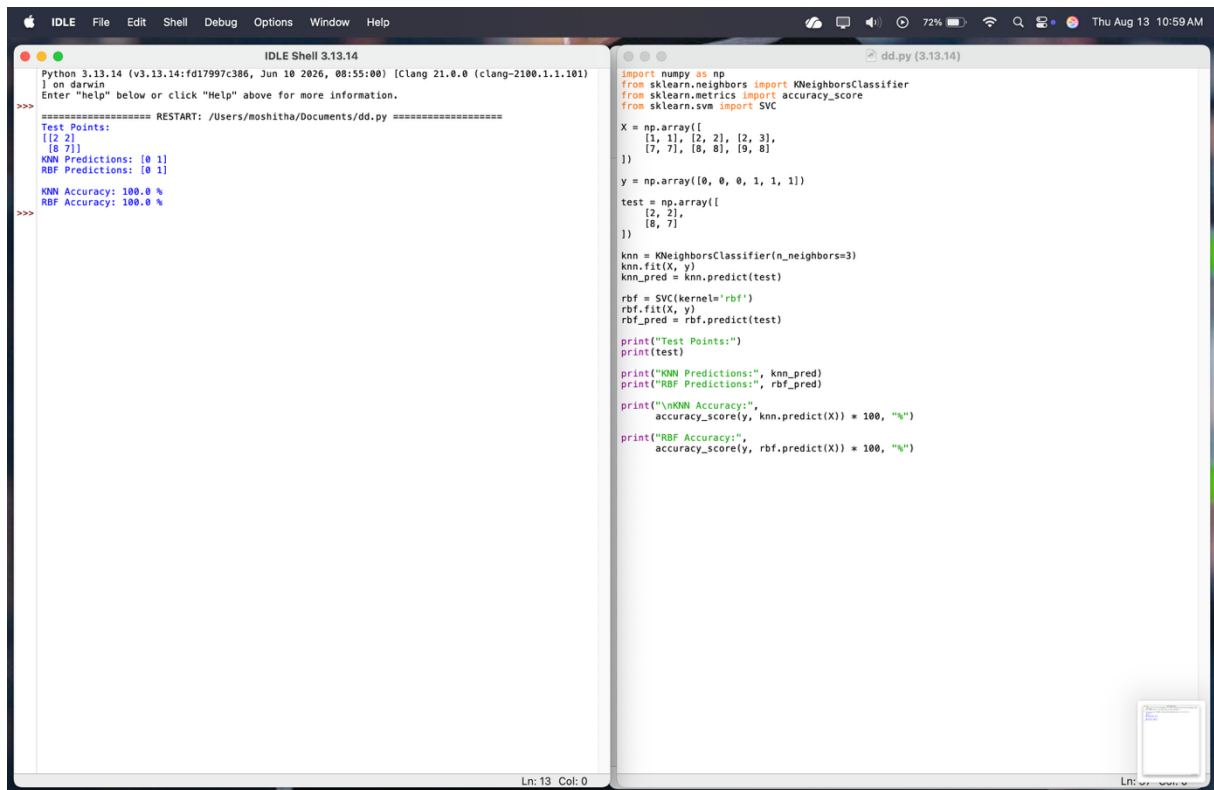
Comparison

KNN:

- Uses nearest neighbours.
- Requires distance calculation during prediction.
- Has little training cost.
- Prediction can be slower for large datasets.
- Sensitive to the value of K.

RBF:

- Uses Gaussian basis functions.
- Builds a model during training.
- Prediction is generally faster after training.
- Depends on centres and widths.
- Can model non-linear relationships.



```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
>>> on darwin
>>> Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/mosh1tha/Documents/dd.py =====
>>>
Test Points:
[[2 2]
 [8 7]]
KNN Predictions: [0 1]
RBF Predictions: [0 1]
KNN Accuracy: 100.0 %
RBF Accuracy: 100.0 %
>>>
```

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC

X = np.array([
    [1, 1], [2, 2], [2, 3],
    [7, 7], [8, 8], [8, 8]
])

y = np.array([0, 0, 0, 1, 1, 1])

test = np.array([
    [2, 2],
    [8, 7]
])

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)
knn_pred = knn.predict(test)

rbf = SVC(kernel='rbf')
rbf.fit(X, y)
rbf_pred = rbf.predict(test)

print("Test Points:")
print(test)

print("KNN Predictions:", knn_pred)
print("RBF Predictions:", rbf_pred)

print("\nKNN Accuracy:",
      accuracy_score(y, knn.predict(X)) * 100, "%")

print("RBF Accuracy:",
      accuracy_score(y, rbf.predict(X)) * 100, "%")
```

Analysis

KNN is simple and effective for small datasets. RBF models can provide smoother non-linear decision boundaries but require proper selection of centres and widths.

Result

Thus, both KNN and RBF can perform classification, but they differ in their learning process, computational requirements, and prediction behaviour.

6. Write a Python program to implement the Decision Tree algorithm for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.

Aim

To implement a Decision Tree algorithm for classification using Gini Index or Information Gain and visualize the tree structure.

Concept

A Decision Tree is a supervised learning algorithm that recursively divides the dataset based on feature values. The feature providing the best split is selected at each node.

Gini Index Formula

$$\text{Gini} = 1 - \sum p_i^2$$

where p_i is the probability of a class.

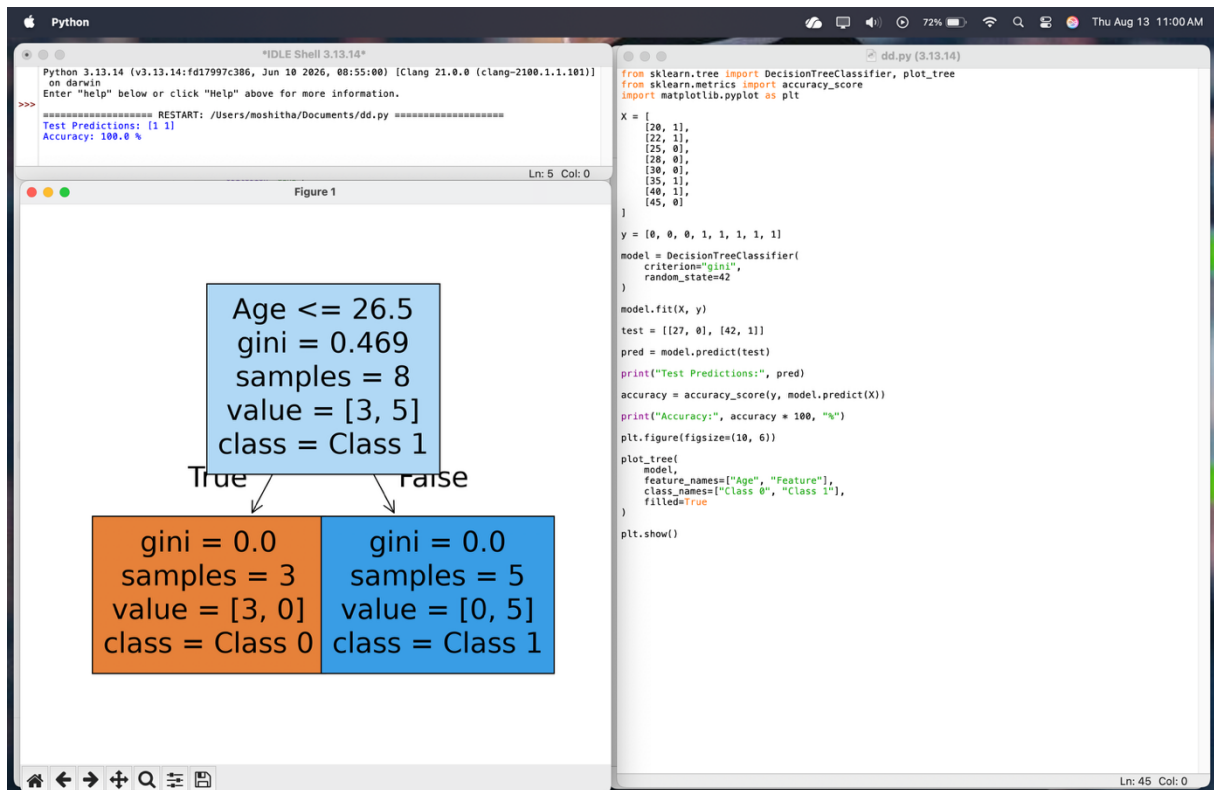
Information Gain

Information Gain measures the reduction in entropy after splitting the dataset.

$$\text{Information Gain} = \text{Entropy before split} - \text{Entropy after split}$$

Algorithm

1. Prepare the training dataset.
2. Calculate impurity using Gini Index or Information Gain.
3. Evaluate possible feature splits.
4. Select the feature giving the best split.
5. Divide the dataset.
6. Repeat the process for each branch.
7. Stop when the stopping condition is satisfied.
8. Use the constructed tree for prediction.
9. Calculate classification accuracy.
10. Visualize the tree structure.



Analysis

Feature selection directly affects the structure and performance of the Decision Tree. Selecting an informative feature at the root generally results in better classification.

Result

Thus, the Decision Tree successfully classifies test instances and represents the classification process in an easy-to-understand tree structure.

7. Develop a Python program to implement a Naïve Bayes classifier for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

Aim

To implement a Naïve Bayes classifier, calculate prior and conditional probabilities, and compare classification with and without Laplace smoothing.

Concept

Naïve Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that features are conditionally independent given the class.

Bayes Formula

$$P(C|X) = P(X|C)P(C) / P(X)$$

For classification:

$$P(C|X) \propto P(C) \times \prod P(x_i|C)$$

Laplace Smoothing Formula

$$P(x|C) = (\text{count}(x,C) + 1) / (\text{count}(C) + \text{number of possible values})$$

Algorithm

1. Prepare the dataset.
2. Calculate the prior probability of each class.
3. Calculate conditional probabilities of features.
4. Multiply the prior and conditional probabilities.
5. Select the class with the highest probability.
6. Repeat classification for test instances.
7. Calculate accuracy.
8. Apply Laplace smoothing.
9. Compare results with and without smoothing.

The screenshot shows an IDE with two panels. The left panel is the IDLE Shell, displaying the output of a Python script. The right panel is the editor, showing the source code of the script. The script defines a Naïve Bayes classifier with Laplace smoothing. It uses a dictionary to store class counts and calculates probabilities for each class. The test instance is 'Sunny', 'Cold'.

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 00:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/moshitha/Documents/dd.py =====
Test Instance: ('Sunny', 'Cold')
With Laplace Smoothing: Yes
Without Laplace Smoothing: Yes
>>>
```

```
X = [
    ("Sunny", "Hot"),
    ("Sunny", "Hot"),
    ("Rainy", "Cold"),
    ("Rainy", "Cold"),
    ("Sunny", "Cold"),
    ("Cloudy", "Cold")
]

y = ["No", "No", "Yes", "Yes", "Yes", "Yes"]

def predict(test, smoothing=True):
    classes = set(y)
    scores = {}

    for c in classes:
        class_count = y.count(c)
        prior = class_count / len(y)

        score = prior

        for feature_index in range(2):
            count = sum(
                1 for i in range(len(X))
                if y[i] == c and X[i][feature_index] == test[feature_index]
            )

            if smoothing:
                probability = (count + 1) / (class_count + 3)
            else:
                probability = count / class_count

            score *= probability

        scores[c] = score

    return max(scores, key=scores.get)

test = ("Sunny", "Cold")

print("Test Instance:", test)
print("With Laplace Smoothing:", predict(test, True))
print("Without Laplace Smoothing:", predict(test, False))
```

Analysis

Without smoothing, an unseen feature may produce zero probability and affect the entire classification. Laplace smoothing prevents zero probabilities and makes the classifier more robust.

Result

Thus, the Naïve Bayes classifier successfully predicts class labels, and Laplace smoothing helps handle unseen feature values.

8. Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Aim

To implement Logistic Regression for binary classification using gradient descent.

Concept

Logistic Regression is a supervised classification algorithm that predicts the probability of a binary class using the sigmoid function.

Sigmoid Function

$$\sigma(z) = 1 / (1 + e^{-z})$$

The sigmoid output lies between 0 and 1.

Classification Rule

If probability ≥ 0.5 , classify as Class 1.

If probability < 0.5 , classify as Class 0.

Algorithm

1. Initialize weights and bias.
2. Calculate the linear combination of inputs.
3. Apply the sigmoid function.
4. Calculate prediction error.
5. Calculate gradients.
6. Update weights and bias using the learning rate.
7. Repeat the process for several iterations.
8. Generate final predictions.
9. Calculate accuracy.

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 00:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/meshitha/Documents/dd.py =====
Weights: 1.784
Bias: -5.69
Sigmoid Outputs:
[0.018 0.693 0.359 0.755 0.944 0.989]
Predicted Classes:
[0 0 1 1]
Accuracy: 100.0 %
>>>
```

```
import numpy as np

X = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y = np.array([0, 0, 0, 1, 1, 1], dtype=float)

w = 0.0
b = 0.0
learning_rate = 0.1

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for epoch in range(1000):
    z = w * X + b
    pred = sigmoid(z)

    dw = np.mean((pred - y) * X)
    db = np.mean(pred - y)

    w -= learning_rate * dw
    b -= learning_rate * db

probabilities = sigmoid(w * X + b)
predicted = (probabilities >= 0.5).astype(int)

accuracy = np.mean(predicted == y) * 100

print("Weights:", round(w, 3))
print("Bias:", round(b, 3))
print("Sigmoid Outputs:")
print(np.round(probabilities, 3))
print("Predicted Classes:")
print(predicted)
print("Accuracy:", round(accuracy, 2), "%")
```

Analysis

The learning rate controls the size of each weight update. A very small learning rate results in slow convergence, while a very large learning rate can cause unstable or overshooting behaviour. A suitable learning rate provides faster and stable convergence.

Result

Thus, Logistic Regression successfully performs binary classification using gradient descent and sigmoid probability outputs.

9. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Aim

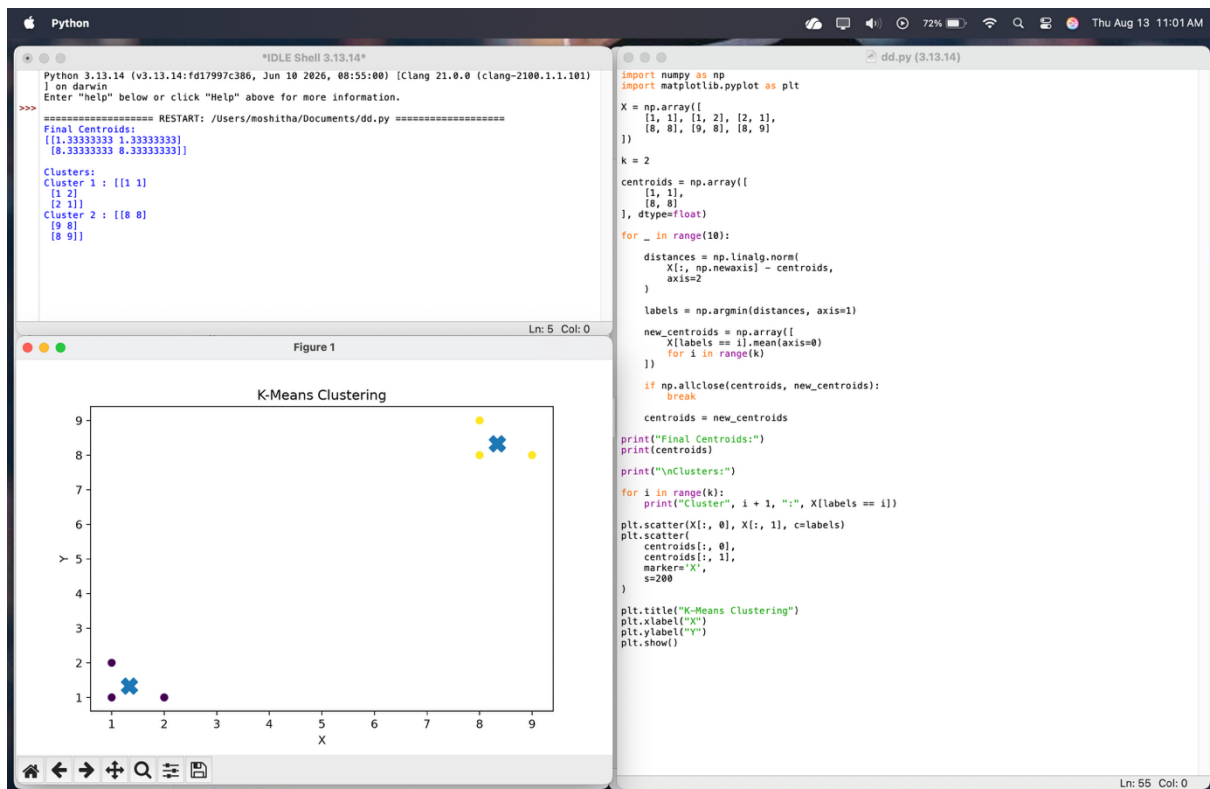
To implement K-Means clustering by assigning data points to clusters and updating centroids iteratively.

Concept

K-Means is an unsupervised learning algorithm that divides data into K clusters. Each data point is assigned to the nearest centroid.

Algorithm

1. Select the number of clusters K .
2. Initialize K cluster centroids.
3. Calculate the distance between each data point and every centroid.
4. Assign each point to the nearest centroid.
5. Recalculate the centroid of each cluster.
6. Repeat assignment and centroid update.
7. Stop when centroids become stable or the maximum number of iterations is reached.
8. Display the final clusters and centroids.
9. Visualize the clusters.



Analysis

The value of K determines the number of clusters. Different initial centroids can lead to different clustering results. Good initialization can improve convergence and clustering quality.

Result

Thus, K-Means successfully groups similar data points into clusters and identifies their final centroids.

10. Write a Python program to implement a simple Genetic Algorithm to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation

operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

Aim

To implement a Genetic Algorithm to optimize a mathematical function using selection, crossover, and mutation.

Concept

A Genetic Algorithm is an optimization technique inspired by natural evolution. It maintains a population of candidate solutions and improves them over generations.

Main Operations

1. Selection
2. Crossover
3. Mutation
4. Fitness evaluation

Algorithm

1. Initialize a population of random solutions.
2. Calculate the fitness of each solution.
3. Select the best individuals as parents.
4. Perform crossover to generate offspring.
5. Apply mutation to some offspring.
6. Create a new population.
7. Evaluate the new population.
8. Repeat the process for several generations.
9. Select the best solution from the final population.
10. Display the best solution and fitness value.

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 00:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/meshitha/Documents/dd.py =====
Generation: 1 Best: 24 Fitness: 576
Generation: 2 Best: 25 Fitness: 625
Generation: 3 Best: 25 Fitness: 625
Generation: 4 Best: 28 Fitness: 784
Generation: 5 Best: 28 Fitness: 784
Generation: 6 Best: 29 Fitness: 841
Generation: 7 Best: 29 Fitness: 841
Generation: 8 Best: 29 Fitness: 841
Generation: 9 Best: 29 Fitness: 841
Generation: 10 Best: 31 Fitness: 961
Generation: 11 Best: 31 Fitness: 961
Generation: 12 Best: 31 Fitness: 961
Generation: 13 Best: 31 Fitness: 961
Generation: 14 Best: 31 Fitness: 961
Generation: 15 Best: 31 Fitness: 961
Generation: 16 Best: 31 Fitness: 961
Generation: 17 Best: 31 Fitness: 961
Generation: 18 Best: 31 Fitness: 961
Generation: 19 Best: 31 Fitness: 961
Generation: 20 Best: 31 Fitness: 961
Best Solution: 31
Best Fitness: 961
>>>

import random
POP_SIZE = 10
GENERATIONS = 20
MUTATION_RATE = 0.1

population = [
    random.randint(0, 31)
    for _ in range(POP_SIZE)
]

def fitness(x):
    return x * x

for generation in range(GENERATIONS):
    population.sort(
        key=fitness,
        reverse=True
    )

    best = population[0]

    print(
        "Generation:", generation + 1,
        "Best:", best,
        "Fitness:", fitness(best)
    )

    parents = population[:4]
    new_population = parents.copy()

    while len(new_population) < POP_SIZE:
        p1 = random.choice(parents)
        p2 = random.choice(parents)

        crossover_point = random.randint(0, 4)
        mask = (1 << crossover_point) - 1
        child = (p1 & mask) | (p2 & ~mask)

        if random.random() < MUTATION_RATE:
            bit = random.randint(0, 4)
            child ^= (1 << bit)

        child = max(0, min(31, child))
        new_population.append(child)

    population = new_population

    best = max(population, key=fitness)
    print("\nBest Solution:", best)
    print("Best Fitness:", fitness(best))
```

Analysis

Important parameters include population size, mutation rate, crossover rate, and number of generations. A larger population provides greater diversity, while excessive mutation may slow convergence. More generations generally provide more opportunities for improvement.

Result

Thus, the Genetic Algorithm successfully searches for an optimal solution by evolving the population over multiple generations.

11. Write a Python program to implement the Support Vector Machine (SVM) algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

Aim

To implement a Support Vector Machine with a linear kernel for binary classification.

Concept

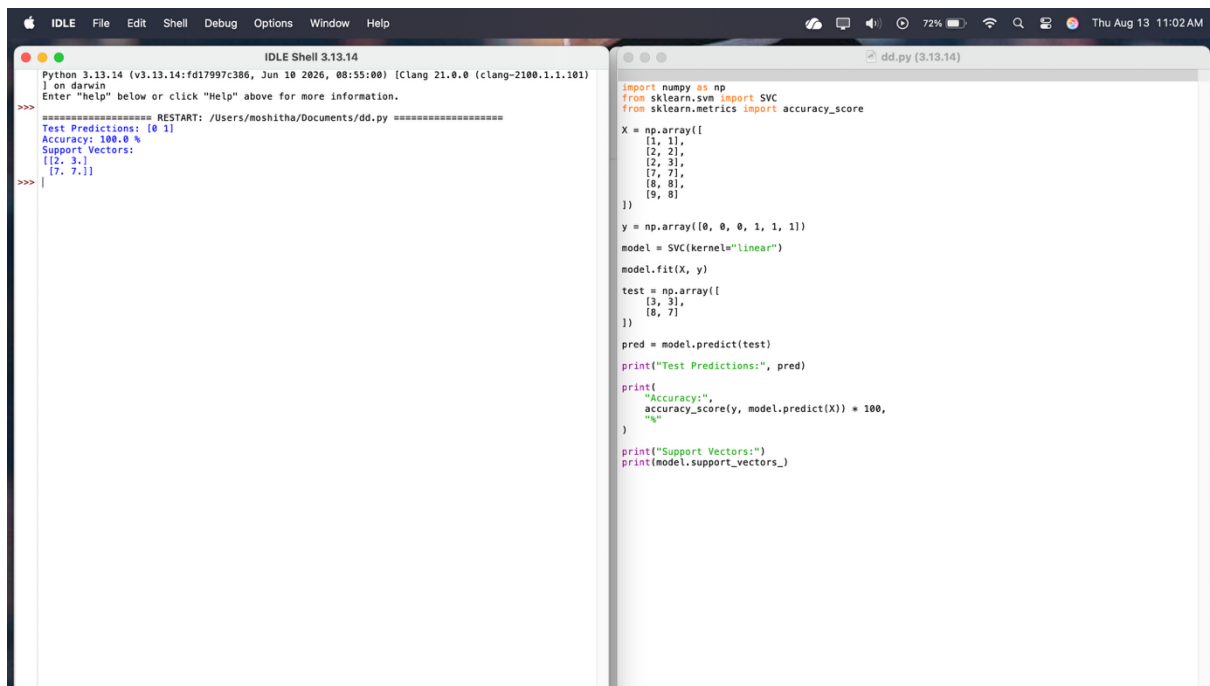
SVM is a supervised learning algorithm that finds an optimal separating hyperplane between classes while maximizing the margin.

Main Concepts

- Hyperplane
- Margin
- Support vectors
- Linear kernel

Algorithm

1. Prepare the binary classification dataset.
2. Select a linear kernel.
3. Train the SVM model.
4. Find the separating hyperplane.
5. Identify support vectors.
6. Classify test instances.
7. Calculate classification accuracy.
8. Analyze the effect of margin and support vectors.



The screenshot shows a Python IDE with two windows. The left window displays the output of a Python script, and the right window displays the source code of the script.

Left Window (Output):

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)]
>>>
===== RESTART: /Users/moshitha/Documents/dd.py =====
Test Predictions: [0 1]
Accuracy: 100.0 %
Support Vectors:
[[2. 3.]
 [7. 7.]]
>>>
```

Right Window (Code):

```
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X = np.array([
    [1, 1],
    [2, 2],
    [2, 3],
    [7, 7],
    [8, 8],
    [9, 8]
])

y = np.array([0, 0, 0, 1, 1, 1])
model = SVC(kernel="linear")
model.fit(X, y)

test = np.array([
    [3, 3],
    [8, 7]
])

pred = model.predict(test)
print("Test Predictions:", pred)

print(
    "Accuracy:",
    accuracy_score(y, model.predict(X)) * 100,
    "%"
)

print("Support Vectors:")
print(model.support_vectors_)
```

Analysis

Support vectors are the data points closest to the decision boundary. They play an important role in determining the position of the hyperplane. A larger margin generally improves the model's ability to generalize to unseen data.

Result

Thus, the SVM successfully classifies binary data by finding a maximum-margin separating hyperplane.

12. Develop a Python program to implement Principal Component Analysis (PCA) for dimensionality reduction. Apply PCA on a dataset, reduce it to two principal components, and visualize the transformed data. Compare model performance before and after dimensionality reduction and analyze variance retention.

Aim

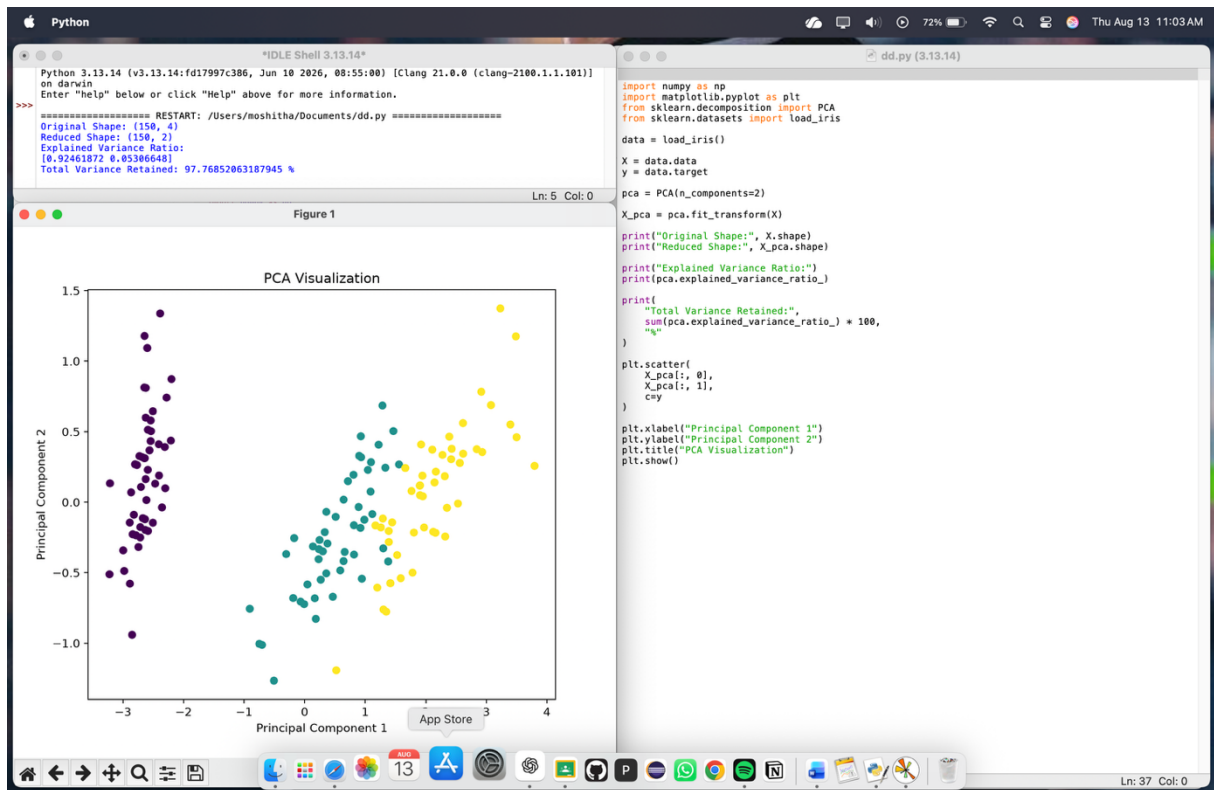
To implement PCA for dimensionality reduction, reduce the dataset to two principal components, and visualize the transformed data.

Concept

PCA is an unsupervised dimensionality reduction technique. It transforms the original features into new uncorrelated principal components that retain maximum variance.

Algorithm

1. Collect the dataset.
2. Standardize the data if required.
3. Calculate the covariance matrix.
4. Calculate eigenvalues and eigenvectors.
5. Sort the principal components according to their eigenvalues.
6. Select the first two principal components.
7. Transform the original data into the reduced space.
8. Visualize the transformed data.
9. Calculate the percentage of retained variance.
10. Compare model performance before and after dimensionality reduction.



Analysis

The first principal component contains the maximum variance, followed by the second component. PCA can reduce computational complexity while retaining important information. However, reducing dimensions too much may result in information loss.

Result

Thus, PCA successfully reduces the dimensionality of the dataset to two principal components while retaining the major portion of the variance.

13. Write a Python program to implement the Hierarchical Clustering algorithm. Use a small dataset and apply different linkage methods such as single, complete, and average linkage. Generate a dendrogram and analyze how cluster formation changes with different linkage criteria.

Aim

To implement Hierarchical Clustering using single, complete, and average linkage methods and generate dendrograms.

Concept

Hierarchical Clustering creates a hierarchy of clusters. The result is represented using a dendrogram.

Linkage Methods

Single Linkage:

Uses the minimum distance between two clusters.

Complete Linkage:

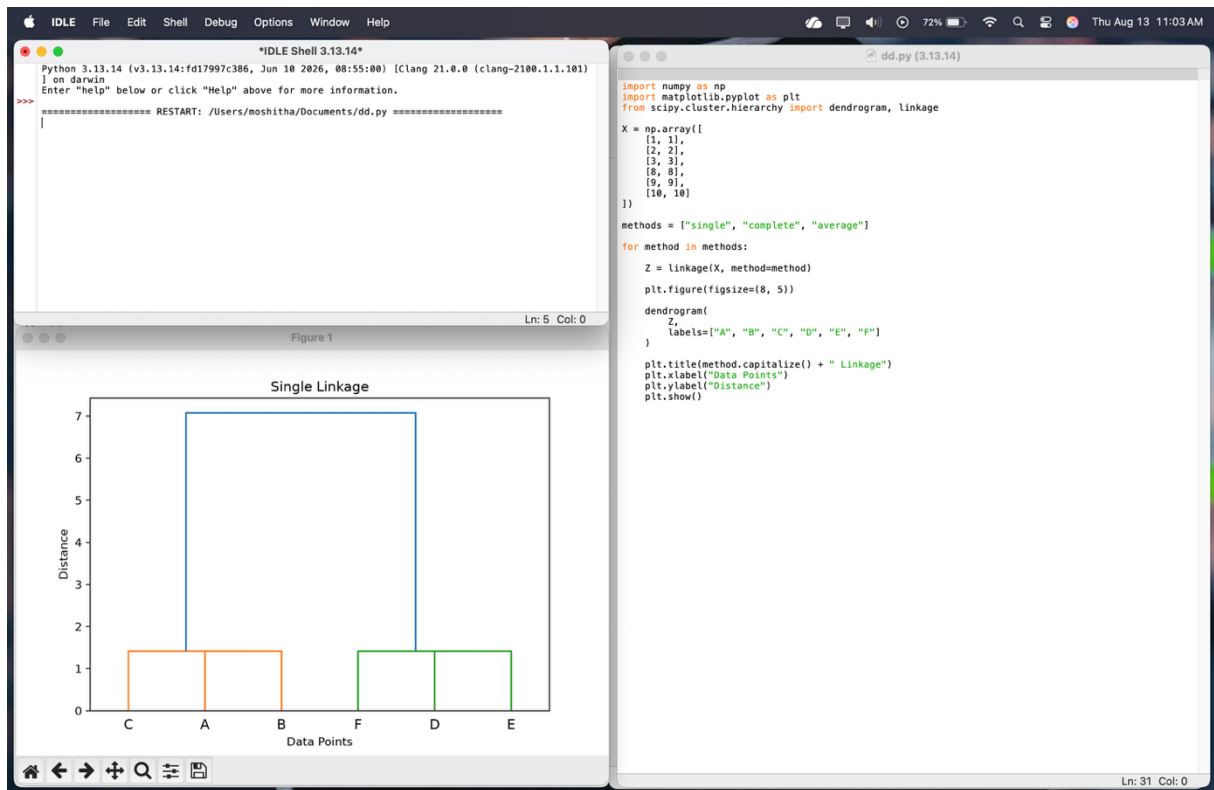
Uses the maximum distance between two clusters.

Average Linkage:

Uses the average distance between all points of two clusters.

Algorithm

1. Prepare the dataset.
2. Calculate pairwise distances.
3. Treat every data point as an individual cluster.
4. Find the closest clusters.
5. Merge the selected clusters.
6. Update the cluster distances.
7. Repeat until all points form one cluster.
8. Generate a dendrogram.
9. Repeat using single, complete, and average linkage.
10. Compare the resulting cluster structures.



Analysis

Different linkage methods produce different cluster formations. Single linkage may create chain-like clusters, complete linkage tends to create compact clusters, and average linkage provides a balance between the two.

Result

Thus, Hierarchical Clustering successfully forms a hierarchy of clusters and demonstrates how linkage criteria influence cluster formation.

14. Create a Python program to implement a simple Artificial Neural Network (ANN) with one hidden layer. Train the network using forward propagation and backpropagation. Test the model on sample inputs and display predicted outputs. Analyze how the number of neurons and learning rate affect performance.

Aim

To implement a simple Artificial Neural Network with one hidden layer using forward propagation and backpropagation.

Concept

An ANN consists of interconnected neurons arranged in layers. The network learns by adjusting weights based on the prediction error.

Main Components

- Input layer
- Hidden layer
- Output layer
- Weights
- Bias
- Activation function

Algorithm

1. Initialize weights and biases randomly.
2. Provide the input data.
3. Perform forward propagation.
4. Calculate the hidden layer output.
5. Calculate the final output.
6. Calculate the prediction error.
7. Perform backpropagation.
8. Calculate gradients.
9. Update weights and biases.
10. Repeat the process for multiple epochs.
11. Test the trained network.
12. Display predicted outputs.

The image shows a screenshot of an IDE (Integrated Development Environment) with two windows. The left window is titled 'IDLE Shell 3.13.14' and displays the output of a Python script. The right window is titled 'dd.py (3.13.14)' and shows the source code of the script.

Left Window Output:

```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/moshitha/Documents/dd.py =====
Predicted Outputs:
[[0.83]
 [0.977]
 [0.973]
 [0.823]]

Predicted Classes:
[[0]
 [1]
 [1]
 [0]]

>>>
```

Right Window Code:

```
import numpy as np

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)

np.random.seed(1)

W1 = np.random.randn(2, 4)
W2 = np.random.randn(4, 1)

learning_rate = 0.5

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

for epoch in range(10000):
    hidden = sigmoid(X @ W1)
    output = sigmoid(hidden @ W2)
    error = y - output
    d_output = error * sigmoid_derivative(output)
    d_hidden = (
        d_output @ W2.T
    ) * sigmoid_derivative(hidden)
    W2 += hidden.T @ d_output * learning_rate
    W1 += X.T @ d_hidden * learning_rate

predicted = sigmoid(
    sigmoid(X @ W1) @ W2
)

print("Predicted Outputs:")
print(np.round(predicted, 3))

print("\nPredicted Classes:")
print((predicted >= 0.5).astype(int))
```

Analysis

The number of neurons in the hidden layer affects the learning capacity of the network. Too few neurons may cause underfitting, while too many neurons may cause overfitting. The learning rate controls how quickly the weights are updated.

Result

Thus, the ANN successfully learns the given pattern through forward propagation and backpropagation.

15. Write a Python program to implement the Apriori algorithm for association rule mining. Use a transaction dataset to generate frequent itemsets and association rules. Display support, confidence, and lift values, and analyze how minimum support and confidence thresholds affect the results.

Aim

To implement the Apriori algorithm for association rule mining and generate frequent itemsets and association rules.

Concept

Apriori is an association rule mining algorithm used to discover relationships among items in transaction datasets.

Support

$\text{Support}(A) = \text{Number of transactions containing } A / \text{Total number of transactions}$

Confidence

$\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A)$

Lift

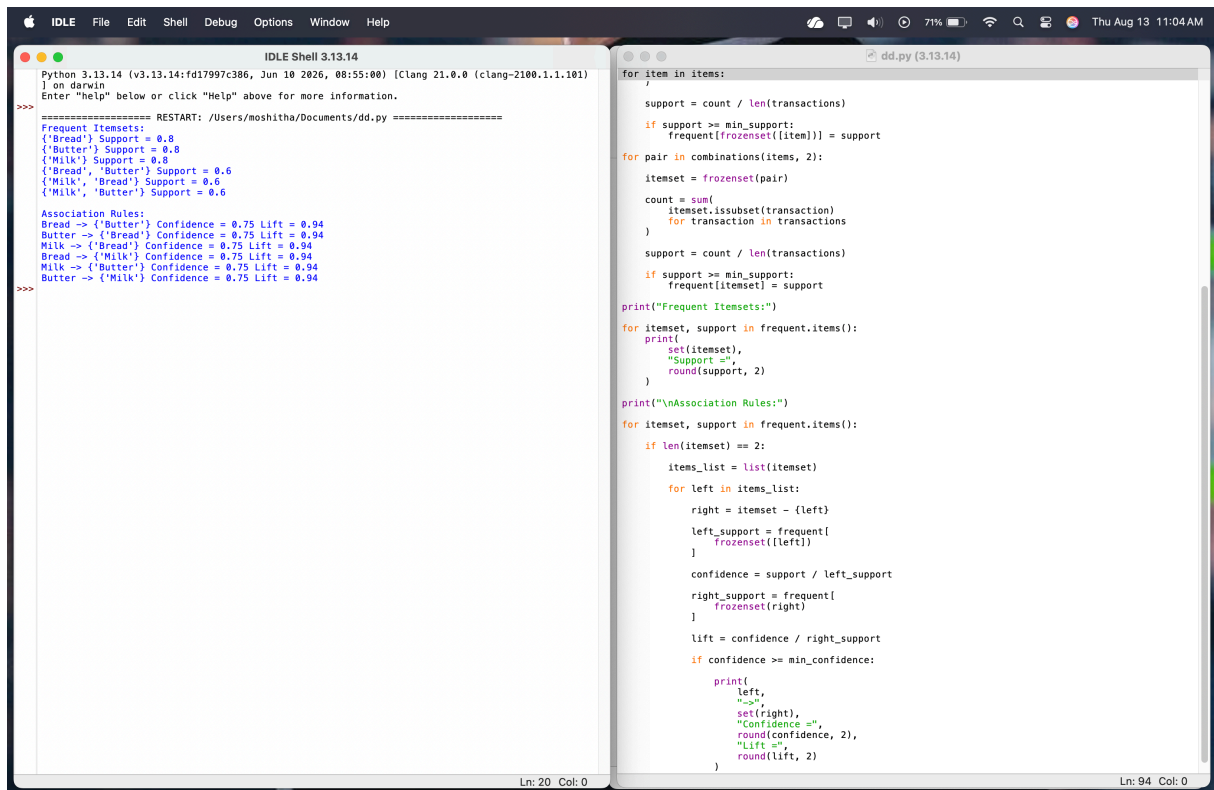
$\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$

Interpretation of Lift

- Lift > 1: Positive association
- Lift = 1: No significant association
- Lift < 1: Negative association

Algorithm

1. Prepare the transaction dataset.
2. Set minimum support.
3. Generate candidate itemsets.
4. Calculate support for each itemset.
5. Remove itemsets below minimum support.
6. Generate larger candidate itemsets.
7. Repeat until no more frequent itemsets can be generated.
8. Generate association rules from frequent itemsets.
9. Calculate support, confidence, and lift.
10. Display the valid association rules.



```
Python 3.13.14 (v3.13.14:fd17997c386, Jun 10 2026, 08:55:00) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/moshitha/Documents/dd.py =====
Frequent Itemsets:
{'Bread'} Support = 0.8
{'Butter'} Support = 0.8
{'Milk'} Support = 0.8
{'Bread', 'Butter'} Support = 0.6
{'Milk', 'Bread'} Support = 0.6
{'Milk', 'Butter'} Support = 0.6

Association Rules:
Bread -> {'Butter'} Confidence = 0.75 Lift = 0.94
Butter -> {'Bread'} Confidence = 0.75 Lift = 0.94
Milk -> {'Bread'} Confidence = 0.75 Lift = 0.94
Bread -> {'Milk'} Confidence = 0.75 Lift = 0.94
Milk -> {'Butter'} Confidence = 0.75 Lift = 0.94
Butter -> {'Milk'} Confidence = 0.75 Lift = 0.94

>>>
```

```
for item in items:
    support = count / len(transactions)
    if support >= min_support:
        frequent[frozenset([item])] = support
for pair in combinations(items, 2):
    itemset = frozenset(pair)
    count = sum(
        itemset.issubset(transaction)
        for transaction in transactions
    )
    support = count / len(transactions)
    if support >= min_support:
        frequent[itemset] = support
print("Frequent Itemsets:")
for itemset, support in frequent.items():
    print(
        set(itemset),
        "Support =",
        round(support, 2)
    )
print("\nAssociation Rules:")
for itemset, support in frequent.items():
    if len(itemset) == 2:
        items_list = list(itemset)
        for left in items_list:
            right = itemset - {left}
            left_support = frequent[
                frozenset([left])
            ]
            confidence = support / left_support
            right_support = frequent[
                frozenset([right])
            ]
            lift = confidence / right_support
            if confidence >= min_confidence:
                print(
                    left,
                    "-> ",
                    set(right),
                    "Confidence =",
                    round(confidence, 2),
                    "Lift =",
                    round(lift, 2)
                )
        )
```

Analysis

A higher minimum support produces fewer frequent itemsets and reduces computation. A lower minimum support produces more itemsets but increases computational requirements. A higher confidence threshold produces fewer but stronger association rules.

Result

Thus, the Apriori algorithm successfully identifies frequent itemsets and generates meaningful association rules using support, confidence, and lift.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand